

Software Technical Specification

aiChair Backend

Obstacle Mapping and Pathing

Version 1

WWU CS Senior Project 2

Spring 2026

Table of Contents

Revision History.....	3
1 Introduction.....	3
2 Technical Specifications: REST API.....	3
2.1 Specification: /health endpoint	3
2.2 Specification: /obstacles endpoint	3
2.3 Specification: /route endpoint	4
3 Technical Specifications: Database.....	5
3.1 Specification: osm_ways and osm_nodes	5
3.2 Specification: obstacles table	6
3.3 Specification: spatial proximity query for nearby obstacles.....	7
3.4 Specification: PGPointCloud schema (planned)	7
4 Technical Specifications: Routing Engine Integration.....	7
4.1 Specification: Unweaver HTTP contract	7
4.2 Specification: Unweaver error response mapping	8
5 Technical Specifications: Input Validation.....	9
5.1 Specification: Bellingham bounding box enforcement.....	9
5.2 Specification: Pydantic request models.....	10
6 Technical Considerations.....	10
6.1 Dependencies	10
6.2 Assumptions	10
6.3 Constraints	11
7 Build and Deployment.....	11
Appendix A Acronyms	11

Revision History

Rev.	Date	Authors	Comments
1	5/22/2026	Ketcham, K. Lounsbery, M.	Initial release.

1 Introduction

This document defines the technical specifications for OMaP (Obstacle Mapping and Pathing), the backend service that powers the aiChair project. aiChair is an accessibility-focused routing app for powered wheelchair users in Bellingham; OMaP is the REST API and database layer that the frontend and the Unweaver routing engine both interact with. Where the Software Requirements Specification captures what the system must do, this document captures how each requirement is implemented in code, what its inputs and outputs look like, and which components it depends on.

Specifications below are grouped by component: the REST API exposed to the frontend, the database schema and spatial query layer, the routing engine integration, and the input validation rules that apply across endpoints. Each specification cites the requirement it satisfies, describes the implementation, and notes any security considerations.

2 Technical Specifications: REST API

OMaP exposes three HTTP endpoints. All requests and responses are JSON, all coordinates use WGS 84, and all input is validated against the Bellingham bounding box (see section 5).

2.1 Specification: /health endpoint

Associated requirement

aiChair Software Requirements Specification, Section 3.1: Liveness check.

Specification details

GET /health returns HTTP 200 with the JSON body {"status":"ok"} when the API process is running. The endpoint performs no database or external service calls; a successful response indicates only that the FastAPI process is reachable.

Security details

Public endpoint. No request body, no parameters, no sensitive information in the response.

2.2 Specification: /obstacles endpoint

Associated requirement

aiChair Software Requirements Specification, Section 3.2: Users can report obstacles encountered on routes (curbs, broken pavement, construction, etc.).

Specification details

POST /obstacles accepts a JSON body and writes a new row to the obstacles table. Request shape:

```
{
  "lat": <float, required>,
  "lon": <float, required>,
  "obstacle_type": <string, 1-50 chars, required>,
  "description": <string, 0-500 chars, optional>
}
```

Pydantic validates that lat and lon are well-formed floats inside the Bellingham bounding box, that obstacle_type is a non-empty string of at most 50 characters, and that description (if present) is at most 500 characters. On success, the endpoint returns HTTP 201 with the inserted row, including the auto-generated id and created_at timestamp.

Status codes

- 201 Created: obstacle inserted successfully.
- 422 Unprocessable Entity: input failed validation (bad coordinates, missing field, length violation, coordinates outside the Bellingham bbox).

Security details

Length limits on obstacle_type and description provide first-pass protection against unbounded input. Database writes go through the backend service only; direct anon-key writes from outside the backend are blocked by Row Level Security policies on the table.

2.3 Specification: /route endpoint**Associated requirement**

aiChair Software Requirements Specification, Section 3.3: Users can request an accessible route between two points, with obstacles along the route returned for display.

Specification details

POST /route accepts a JSON body with start and end coordinates plus a routing profile name, then returns a route as a list of line segments along with any obstacles within 25 metres of the route geometry. Request shape:

```
{
  "lat1": <float, required>,
  "lon1": <float, required>,
  "lat2": <float, required>,
  "lon2": <float, required>,
  "profile": <string, 1-50 chars, required>
}
```

The endpoint performs the following steps in order:

- Validates input via Pydantic, including bbox enforcement on both coordinate pairs (see section 5).
- Calls Unweaver at GET /shortest_path/<profile>.json with the four coordinates as query parameters. The profile is URL-encoded before forwarding.
- Parses Unweaver's response envelope first to detect documented error codes (NoPath, InvalidWaypoint, NoGraph), then parses the success shape (edges with LineString geometry).
- Builds a MULTILINESTRING WKT string from the returned edges and queries the obstacles table with ST_DWithin, casting both sides to geography so the buffer threshold is interpreted in metres.
- Returns a JSON object with segments (array of coordinate lists) and obstacles (array of nearby obstacle records).

Response shape:

```
{
  "segments": [
    { "coordinates": [[<lon>, <lat>], ...] },
    ...
  ],
  "obstacles": [
    { "id": <int>, "lat": <float>, "lon": <float>,
      "obstacle_type": <string>, "description": <string|null> },
    ...
  ]
}
```

Status codes

- 200 OK: route found and returned.
- 404 Not Found: Unweaver returned NoPath. No route exists between the requested points.
- 422 Unprocessable Entity: input validation failed, or Unweaver returned InvalidWaypoint.
- 502 Bad Gateway: Unweaver returned an unexpected response shape or an HTTP error.
- 503 Service Unavailable: Unweaver returned NoGraph. The requested profile is not loaded.
- 504 Gateway Timeout: the Unweaver request exceeded its timeout.

Security details

The profile field is capped at 50 characters and URL-encoded before being placed in the Unweaver request path. Bounding box validation on input coordinates ensures the endpoint only operates within the service area.

3 Technical Specifications: Database

The database is a managed Supabase instance (cloud-hosted Postgres with the PostGIS extension). The backend connects to it directly to read OSM geometry, read obstacles for proximity queries, and write new obstacle reports.

3.1 Specification: osm_ways and osm_nodes

Associated requirement

aiChair Software Requirements Specification, Section 3.4: The system must store OpenStreetMap geometry to support routing within the Bellingham service area.

Specification details

OSM ways and nodes are imported once via the Overpass API and stored in two tables:

```
osm_nodes (
  osm_id    bigint PRIMARY KEY,
  lat       double precision,
  lon       double precision,
  tags      jsonb,
  geom      geometry(Point, 4326)
)

osm_ways (
  osm_id    bigint PRIMARY KEY,
  node_ids  bigint[],
  tags      jsonb,
  geom      geometry(LineString, 4326)
)
```

Both tables have GIST indexes on the geom column to support PostGIS spatial queries.

Security details

Read-only at runtime from the backend. The tables have Row Level Security enabled with no policies, so direct anon-key access from outside the backend is blocked.

3.2 Specification: obstacles table**Associated requirement**

aiChair Software Requirements Specification, Section 3.2: User-reported obstacles must be persisted and made available to subsequent route queries.

Specification details

Obstacle reports are stored in a single table. The geom column is auto-derived from lat and lon by Postgres, so the application only writes lat and lon directly.

```
obstacles (
  id          bigserial PRIMARY KEY,
  lat         double precision NOT NULL,
  lon         double precision NOT NULL,
  obstacle_type text NOT NULL,
  description  text NULL,
  geom        geometry(Point, 4326) GENERATED ALWAYS AS
              (ST_SetSRID(ST_MakePoint(lon, lat), 4326)) STORED,
  created_at  timestamptz NOT NULL DEFAULT now()
)
```

A GIST index on geom (obstacles_geom_idx) supports the spatial proximity query used by the /route endpoint.

Security details

Same as 3.1: RLS enabled with no policies; only the backend's service-role connection can read or write.

3.3 Specification: spatial proximity query for nearby obstacles

Associated requirement

aiChair Software Requirements Specification, Section 3.3.1: Obstacles within a configurable distance of a computed route must be returned alongside the route geometry.

Specification details

When the /route endpoint receives Unweaver's edge list, it converts the edges into a MULTILINESTRING WKT string and runs the following query against the obstacles table:

```
SELECT id, lat, lon, obstacle_type, description
FROM obstacles
WHERE ST_DWithin(
  geom::geography,
  ST_GeomFromText($1, 4326)::geography,
  $2
)
```

Casting both sides to geography means the third parameter is interpreted in metres rather than degrees. The current threshold is 25 metres.

Security details

The query uses parameterised input; the WKT string is built from Unweaver's validated response, not from user input.

3.4 Specification: PGPointCloud schema

Associated requirement

aiChair Software Requirements Specification, Section 3.7: The database should be able to hold LiDAR point cloud data to support future surface-quality and obstacle-detection features.

Specification details

A separate schema using the PGPointCloud Postgres extension is planned for storing LiDAR-derived point clouds of the Bellingham service area.

Security details

Not applicable until the schema is implemented. Expected to follow the same RLS-with-no-policies pattern as the other tables.

4 Technical Specifications: Routing Engine Integration

4.1 Specification: Unweaver HTTP contract

Associated requirement

aiChair Software Requirements Specification, Section 3.3: The backend must compute accessibility-aware routes by delegating to Unweaver, an existing research-grade routing engine.

Specification details

The backend reaches Unweaver over HTTP. The relevant endpoint is:

```
GET /shortest_path/<profile>.json?lon1=..&lat1=..&lon2=..&lat2=..
```

The profile name is part of the URL path and selects which cost function Unweaver uses. The profile field at the FastAPI layer is accepted as a string up to 50 characters; the set of valid profile names is defined by the Unweaver configuration.

Each profile is a Python module on the Unweaver side that assigns a cost to traversing a given OSM way based on the way's tags. For accessibility-aware profiles, this is where wheelchair-specific judgments live: crosswalks can be assigned a higher cost than ordinary sidewalks, stairs can be assigned infinite cost (effectively making them impassable), and surface tags can influence cost depending on the profile. The backend is intentionally unaware of these cost decisions; profile authoring is owned by the Unweaver side of the project.

Unweaver's success response is a JSON object with the following relevant fields:

```
{
  "status": "Ok",
  "origin":    <GeoJSON Feature[Point]>,
  "destination": <GeoJSON Feature[Point]>,
  "total_cost": <float>,
  "edges": [
    {
      "geometry": { "type": "LineString", "coordinates": [...] },
      "properties": { ... }
    },
    ...
  ]
}
```

The backend uses the status and edges fields from this response. Other fields are discarded.

Security details

Unweaver is reachable only from inside the Compose network at hostname unweaver, port 8000, and is not exposed to the public internet. The profile field is URL-encoded before being placed in the request path. No credentials are sent to Unweaver.

4.2 Specification: Unweaver error response mapping

Associated requirement

aiChair Software Requirements Specification, Section 3.3.2: Routing failures must be reported to the frontend with appropriate HTTP status codes and human-readable messages.

Specification details

Unweaver returns documented error codes at HTTP 200 (the request was syntactically valid; routing simply failed). The backend parses an envelope shape first to detect these before attempting to parse the full success response:

```
class UnweaverEnvelope(BaseModel):
    status: str
    code: str | None = None
```

If status is not "Ok", the code field is mapped to an HTTP status that the frontend can act on:

Unweaver code	HTTP status	Backend detail message
NoPath	404 Not Found	No route found between these points
InvalidWaypoint	422 Unprocessable Entity	One or more coordinates can't be routed to
NoGraph	503 Service Unavailable	Routing profile unavailable
(unknown)	502 Bad Gateway	Routing service returned unknown status

Connection-level failures are mapped separately: a request timeout returns 504 Gateway Timeout, and any other transport error returns 502 Bad Gateway.

Security details

Backend detail messages are static and do not echo any portion of Unweaver's raw response, so a misbehaving Unweaver cannot inject content into responses delivered to the frontend.

5 Technical Specifications: Input Validation

5.1 Specification: Bellingham bounding box enforcement

Associated requirement

aiChair Software Requirements Specification, Section 3.5: The backend must reject coordinates outside the Bellingham service area on all endpoints that accept geographic input.

Specification details

The bounding box is defined as a module-level constant:

```
BBOX = (48.68, -122.55, 48.82, -122.35) # south, west, north, east
```

A shared helper `validate_bellingham(lat, lon)` raises `ValueError` if the coordinates fall outside this box. Pydantic field validators on each endpoint's request model invoke this helper at parse time, so any out-of-bbox input is rejected before reaching the database or Unweaver.

For `/obstacles`, the single lat/lon pair is checked. For `/route`, both `(lat1, lon1)` and `(lat2, lon2)` are checked independently.

Security details

Bounding box enforcement limits the database and routing engine's exposed surface to the service area.

5.2 Specification: Pydantic request models

Associated requirement

aiChair Software Requirements Specification, Section 3.6: All incoming HTTP request bodies must be validated against a schema before any business logic runs.

Specification details

Every endpoint that accepts a request body defines a Pydantic BaseModel with explicit field types, length limits, and value ranges. FastAPI invokes Pydantic automatically; a validation failure produces an HTTP 422 response with FastAPI's standard structured error body before the endpoint handler is entered.

Key constraints currently enforced:

- lat: float in $[-90, 90]$, also constrained to the Bellingham bounding box.
- lon: float in $[-180, 180]$, also constrained to the Bellingham bounding box.
- obstacle_type: non-empty string, max 50 characters.
- description: optional string, max 500 characters.
- profile: non-empty string, max 50 characters.

Security details

Length limits provide first-pass protection against unbounded input on text fields. The description field is stored as submitted; the frontend renders it as plain text.

6 Technical Considerations

6.1 Dependencies

Runtime dependencies of the FastAPI service:

- Python 3.13.
- FastAPI for HTTP routing.
- Uvicorn as the ASGI server.
- Pydantic for request and response validation.
- asyncpg for async PostgreSQL connections.
- httpx for the async Unweaver client.

External services required at runtime: Supabase (Postgres with PostGIS) and Unweaver.

6.2 Assumptions

- Frontend-supplied coordinates are in WGS 84. The backend does not perform reprojection.

- Unweaver's response shape matches the contract described in section 4.1.

6.3 Constraints

- The service area is fixed at the Bellingham bounding box. Expanding the area requires reimporting OSM data and updating the bounding-box constant.

7 Build and Deployment

Both services (FastAPI and Unweaver) are built as Docker images and orchestrated by docker-compose.yml at the repo root. A single command (docker compose up --build) builds and starts the full stack. Build automation, dependency management, and first-time setup procedures are documented in the Build Automation User Manual.

Appendix A Acronyms

- API: Application Programming Interface.
- ASGI: Asynchronous Server Gateway Interface.
- GIST: Generalized Search Tree (Postgres index type used for spatial data).
- OSM: OpenStreetMap.
- PostGIS: spatial extension for PostgreSQL.
- REST: Representational State Transfer.
- RLS: Row Level Security (Postgres feature).
- SRS: Software Requirements Specification.
- WGS 84: World Geodetic System 1984, the coordinate reference system used by GPS and OSM (EPSG code 4326).
- WKT: Well-Known Text, a textual representation of geometry used by PostGIS.